互联网接入

每个沙箱都可以访问互联网，同时，可以通过一个公开URL来访问它。

控制互联网访问

您可以在创建沙箱时通过 `allow_internet_access` 参数控制沙箱是否可以访问互联网。默认开放互联网访问（`true`），但您可以在进行安全敏感工作时禁用它。

代码块

```
from inspire_sandbox import Sandbox

# 创建一个支持访问互联网的沙箱(默认)
sandbox = Sandbox.create(allow_internet_access=True)

# 创意一个不可访问互联网的沙箱
isolated_sandbox = Sandbox.create(allow_internet_access=False)
```

当互联网访问被禁用时，沙箱无法创建出站的网络连接，这为敏感代码的执行提供了额外的安全层。

> ❗ 将 `allowInternetAccess` 设置为 `false` 相当于将 `network.denyOut` 设置为 `['0.0.0.0/0']`（拒绝所有流量）。

细粒度网络控制

要对网络访问进行更精细的控制，您可以使用 `network` 配置选项来指定出站流量的允许和拒绝列表。

允许和拒绝列表

您可以指定允许沙箱使用的IP地址、CIDR块或域名：

代码块

```
from inspire_sandbox import Sandbox, ALL_TRAFFIC

# 除了特定IP外，拒绝所有流量
sandbox = Sandbox.create(
    network={
```

```
        "deny_out": [ALL_TRAFFIC],
        "allow_out": ["1.1.1.1", "8.8.8.0/24"]
    }
)

# 仅拒绝特定IP
restricted_sandbox = Sandbox.create(
    network={
        "deny_out": ["8.8.8.8"]
    }
)
```

基于域名的过滤

您可以通过在 `allow out` 中指定主机名来允许流向特定域名的流量。使用基于域名的过滤时，必须在 `deny out` 中包含 `ALL_TRAFFIC` 以阻止所有其他流量。 `deny out` 列表中不支持域名。

代码块

```
from inspire_sandbox import Sandbox, ALL_TRAFFIC

# 仅允许到baidu.com的流量
sandbox = Sandbox.create(
    network={
        "allow_out": ["baidu.com"],
        "deny_out": [ALL_TRAFFIC]
    }
)
```

> 使用任何域名时，都会自动允许默认的名称服务器 `8.8.8.8` 以确保正确的 DNS 解析。

您还可以使用通配符来允许域的所有子域：

代码块

```
from inspire_sandbox import Sandbox, ALL_TRAFFIC

# 允许到mydomain.com的子域名的任何流量
sandbox = Sandbox.create(
    network={
        "allow_out": ["*.mydomain.com"],
        "deny_out": [ALL_TRAFFIC]
    }
)
```

您可以结合使用 域名、IP 地址和CIDR 块：

代码块

```
from inspire_sandbox import Sandbox, ALL_TRAFFIC

# 允许到特定域名和IP的所有流量
sandbox = Sandbox.create(
    network={
        "allow_out": ["api.example.com", "*.github.com", "8.8.8.8"],
        "deny_out": [ALL_TRAFFIC]
    }
)
```

❗ 基于域名的过滤仅适用于端口 80 上的 HTTP 流量（通过Host Header检查）和端口 443 上的 TLS 流量（通过 SNI 检查）。其他端口上的流量仅使用基于 CIDR 的过滤。域名过滤不支持基于 UDP 的协议（例如 QUIC/HTTP3）。

优先级规则

当同时指定 `allow out` 和 `deny out` 时，**`允许` 规则始终优先于 `拒绝`** 规则。这意味着如果某个 IP 地址同时存在于两个列表中，则该 IP 地址将被允许。

代码块

```
from inspire_sandbox import Sandbox, ALL_TRAFFIC

# 尽管拒绝了所有流量(ALL_TRAFFIC), 1.1.1.1和8.8.8.8被精确地允许
sandbox = Sandbox.create(
    network={
        "deny_out": [ALL_TRAFFIC],
        "allow_out": ["1.1.1.1", "8.8.8.8"]
    }
)
```

ALL_TRAFFIC 快捷常量

`ALL_TRAFFIC` 常量表示 CIDR 范围 `0.0.0.0/0`，它匹配所有 IP 地址。使用它可以轻松拒绝或允许所有网络流量：

代码块

```
from inspire_sandbox import Sandbox, ALL_TRAFFIC

```

```
# 拒绝所有的出站流量
sandbox = Sandbox.create(
    network={
        "deny_out": [ALL_TRAFFIC]
    }
)
```

沙箱公开URL

每个沙箱都有一个公开URL，可用于访问沙箱内正在运行的服务。

代码块

```
from inspire_sandbox import Sandbox

sandbox = Sandbox.create()

# 你总是需要给出一个端口号以获取主机名
host = sandbox.get_host(3000)
print(f'https://{host}')
```

上面的代码将打印类似如下内容：

代码块

```
https://sbx-3000-i717t1t7xvu4yh5b39lce.inspire.cn
```

主机名的数字部分是我们传递给该方法的端口号。

限制到沙箱链接的外部访问

默认情况下，沙箱链接可被公开地访问。您可以使用 `allowPublicTraffic` 选项，要求身份验证以限制访问：

代码块

```
from inspire_sandbox import Sandbox

# 创建限制外部访问的沙箱
sandbox = Sandbox.create(
    network={
        "allow_public_traffic": False
    }
```

```python
)

# 这沙箱有一个流量访问令牌
print(sandbox.traffic_access_token)

# 在沙箱内启动一个服务器
sandbox.commands.run("python -m http.server 8080", background=True)

host = sandbox.get_host(8080)
url = f"https://{host}"

# 不带token的访问会得到403
response1 = requests.get(url)
print(response1.status_code) # 403

# 带token的访问会成功
response2 = requests.get(url, headers={
    'sbx-traffic-access-token': sandbox.traffic_access_token
})
print(response2.status_code)  # 200
```

当 `allowPublicTraffic` 被设置为 `false` 时，对沙箱公开URL的所有请求都必须包含 `sbx-traffic-access-token` 标头以及来自 `sandbox.trafficAccessToken` 的值。

连接到沙箱内运行的服务器

您可以在沙箱内启动服务器并使用上述方法连接到它。

在此示例中，我们将启动一个简单的 HTTP 服务器，该服务器侦听端口 3000 并以服务器启动目录的内容进行响应。

代码块

```python
import request
from inspire_sandbox import Sandbox

sandbox = Sandbox.create()

# 在沙箱内启动一个简单的HTTP服务器
process = sandbox.commands.run("python -m http.server 3000", background=True)
host = sandbox.get_host(3000)
url = f"https://{host}"
print('Server started at:', url)

# 从沙箱内服务器获取数据
response = requests.get(url)
```

```
data = response.text
print('Response from server inside sandbox:', data)

# 杀停沙箱内的服务器进程
process.kill()
```

输出大概长这样：

代码块

```
Server started at: https://sbx-3000-i717t1t7xvu4yh5b39lce.inspire.cn
Response from server inside sandbox: <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8">
<title>Directory listing for /</title>
</head>
<body>
<h1>Directory listing for /</h1>
<hr>
<ul>
<li><a href=".bash_logout">.bash_logout</a></li>
<li><a href=".bashrc">.bashrc</a></li>
<li><a href=".profile">.profile</a></li>
</ul>
<hr>
</body>
</html>
```

遮掩请求的Host标头

您可以使用 `maskRequestHost` 选项定制发送到沙箱内运行的服务器的 `Host` 标头。当您的应用程序需要特定的Host格式时，这非常有用。

代码块

```
from inspire_sandbox import Sandbox

# 创建带有定制Host掩码的沙箱
sandbox = Sandbox.create(
    network={
        "mask_request_host": "localhost:${PORT}"
    }
)
```

```

# ${PORT}变量会被替换为特定的端口号
# 到沙箱的请求将会带有被设置为比如 localhost:8080 的 Host 标头
```

掩码中的 `${PORT}` 变量将自动替换为被请求服务的实际端口号。